

# **Linux - Linha de Comando**

Laboratório de Fundamentos em  
Tecnologia da Informação – LFT

# Redirecionamento

- Uma das vantagens da linha de comando Linux é o redirecionamento de entrada e saída.
- Este recurso redireciona a saída padrão de um programa/comando/script para algum dispositivo ou arquivo ao invés do dispositivo de saída padrão (tela). Quando é usado com arquivos, este redirecionamento cria ou substitui o conteúdo do arquivo.

# Redirecionamento

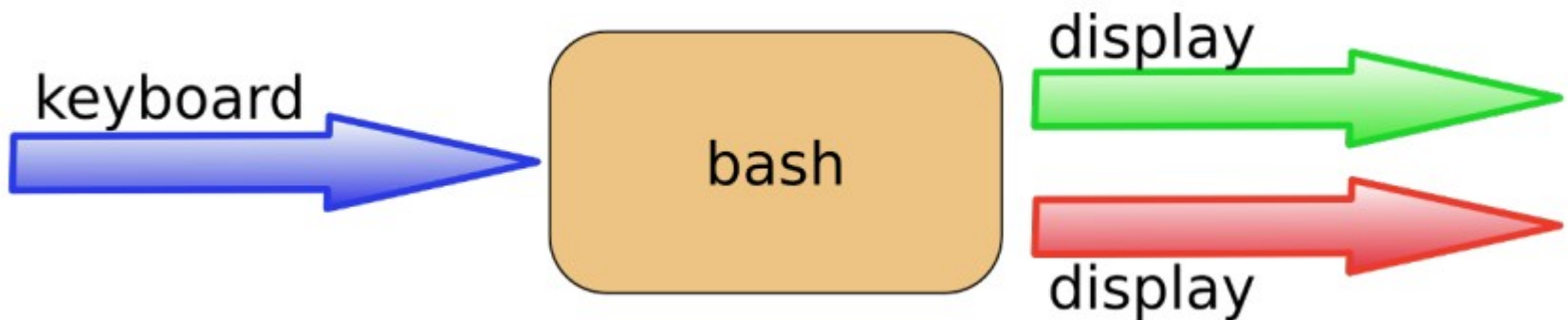
- Por padrão, a entrada do Shell é o teclado, a saída, a tela, e os erros são exibidos na tela também.
- **Entrada de dados:** representada por *stdin* (0);
- **Saída de dados:** representada por *stdout* (1);
- **Saída de erros:** representada por *stderr* (2);

# Redirecionamento

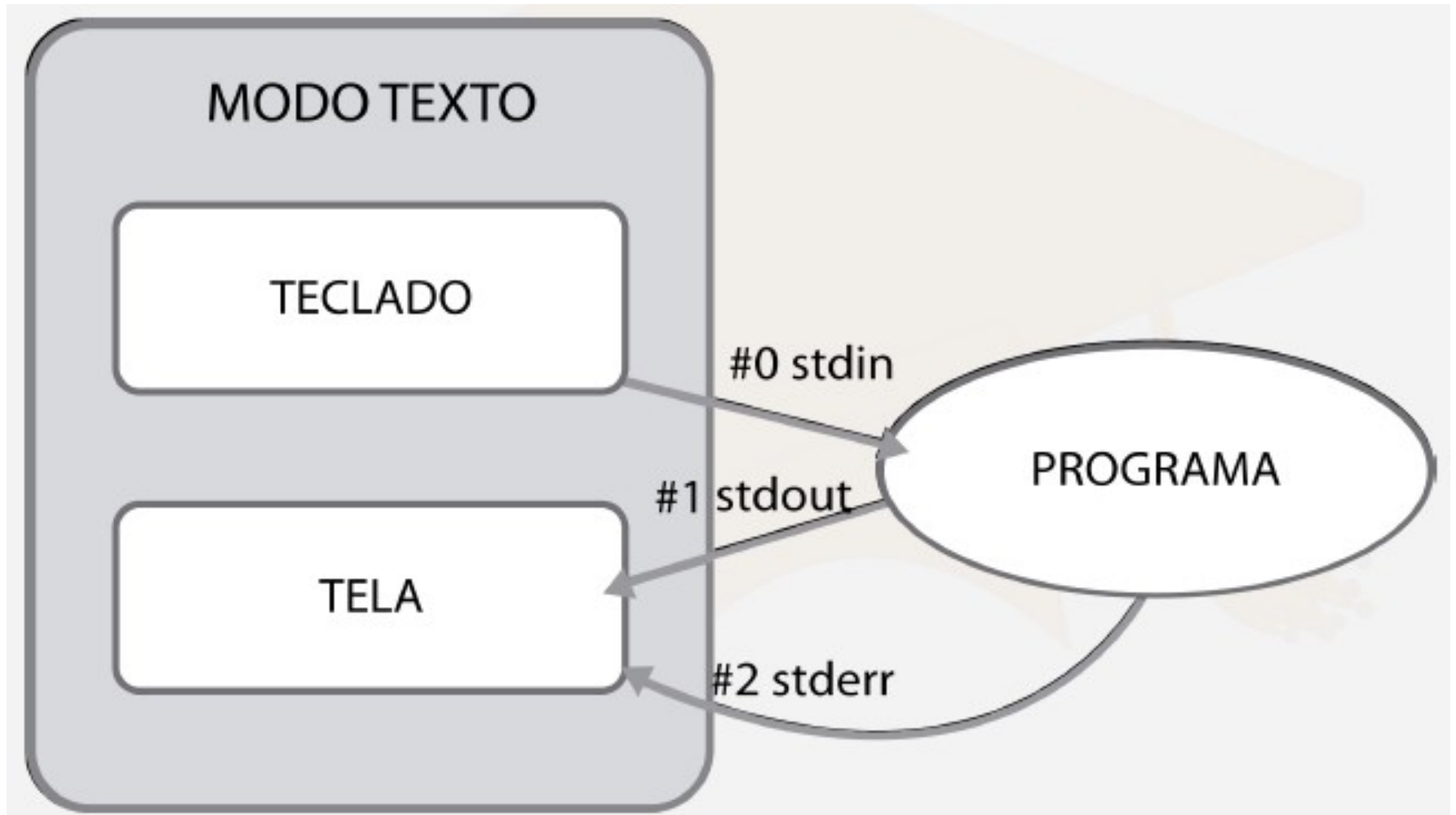
- **Shell bash**
- **Streams: stdin, stdout, and stderr**
  - Stream é uma abstração para controle do fluxo de dados de entrada e saída.
  - **stdin** (stream **0**) recebe dados de entrada, dados de saída são enviados para **stdout** (stream **1**) e mensagens de erro vão para **stderr** (stream **2**)

# Redirecionamento

- **stdin**
  - Geralmente associado ao teclado.
- **stdout**
  - Envia dados para o vídeo.
- **stderr**
  - Envia dados para o vídeo.

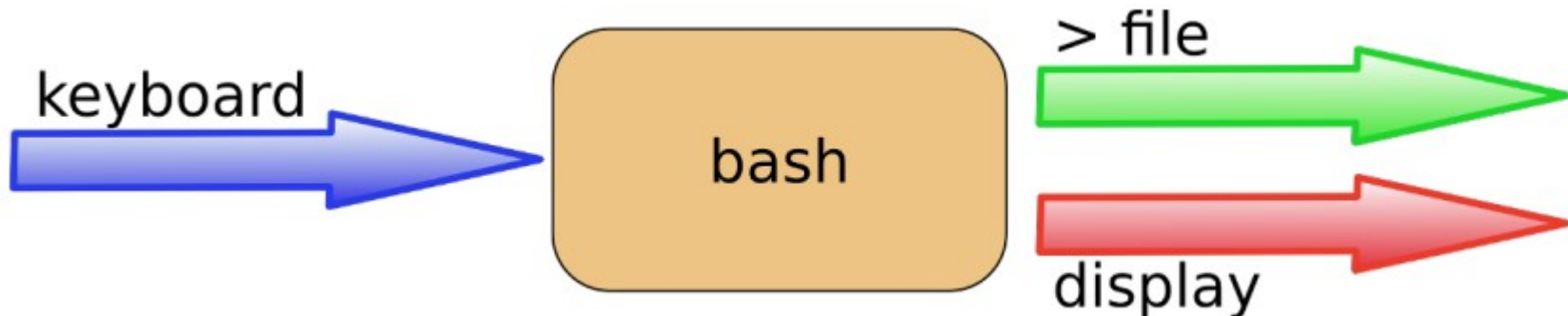


# Redirecionamento



# Redirecionamento

- **Redirecionando dados para saída 1>**
  - Use o sinal “maior que” >. A notação > é a mesma que 1> (**stdout** referenciado como stream **1**).
  - fulano@pc01:~\$ **echo Hoje está chovendo.**  
**Hoje está chovendo.**  
fulano@pc01:~\$ **echo Hoje está chovendo. > tempo.txt**  
fulano@pc01:~\$ **cat tempo.txt**  
**Hoje está chovendo.**  
Quando Shell encontra o sinal de > o conteúdo de **file** é apagado.



# Redirecionamento

- **Redirecionando dados para saída 1>**

- O arquivo (*file*) é sempre apagado quando o Shell encontra o >. Porém isto acontece antes da execução do comando (argumento 0), que abaixo é “zcho”. Ou seja, o arquivo “tempo.txt” é apagado antes de verificar se “zecho” é um comando válido.

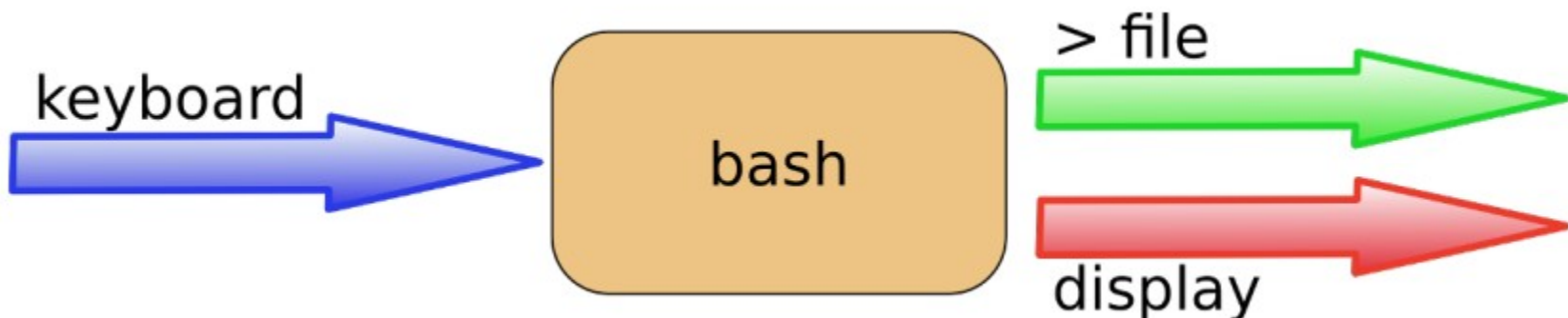
- fulano@pc01:~\$ **cat tempo.txt**

**Hoje está chovendo.**

fulano@pc01:~\$ **zcho Hoje está chovendo. > tempo.txt**

bash: zcho: command not found

fulano@pc01:~\$ **cat tempo.txt**



# Redirecionamento

- **Redirecionando dados para saída 1>**

- A ação de apagar o conteúdo anterior do arquivo (*file*) pode ser prevenida. Use a opção **noclobber**.

- fulano@pc01:~\$ **cat tempo.txt**

**Hoje está chovendo.**

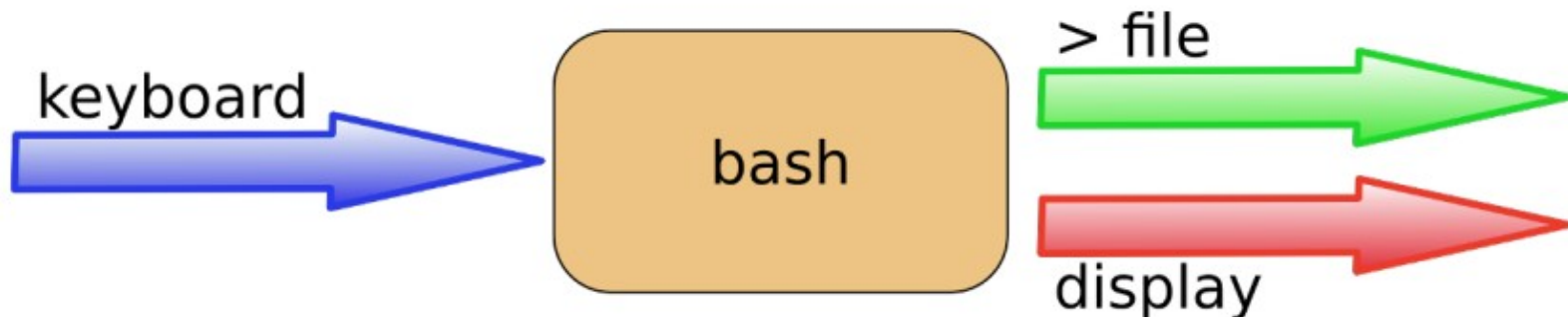
fulano@pc01:~\$ **set -o noclobber**

fulano@pc01:~\$ **echo Hoje está chovendo. > tempo.txt**

-bash: tempo.txt: cannot overwrite existing file

fulano@pc01:~\$ **set +o noclobber**

- Neste caso o arquivo tempo não foi modificado.



# Redirecionamento

- **Acrescentando dados ao final de arquivo**

- O sinal >> redireciona os dados para o final do arquivo especificado.

- fulano@pc01:~\$ **cat tempo.txt**

- fulano@pc01:~\$ **echo Hoje está chovendo. > tempo.txt**

- fulano@pc01:~\$ **cat tempo.txt**

- Hoje está chovendo.

- fulano@pc01:~\$ **echo Mas ontem não choveu. >> tempo.txt**

- fulano@pc01:~\$ **cat tempo.txt**

- Hoje está chovendo.

- Mas ontem não choveu.**

- Os dados foram acrescentados ao final do arquivo e o conteúdo anterior foi mantido.

# Redirecionamento

- **Redirecionando erros 2>**

- Útil para que os erros não apareçam na tela.

- Jogando fora os erros:

- `fulano@pc01:~$ find / > allfiles.txt 2> /dev/null`

- `fulano@pc01:~$ cat tempo.txt`

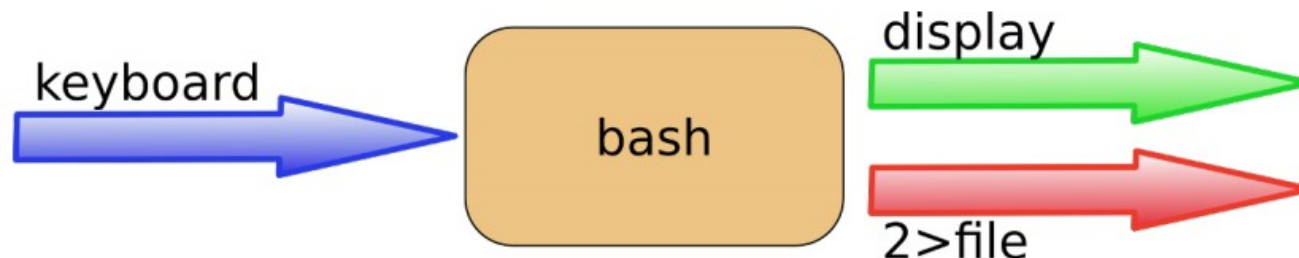
Hoje está chovendo.

`fulano@pc01:~$ zcho Hoje está chovendo. 2> tempo.txt`

`fulano@pc01:~$ cat tempo.txt`

Comando 'zcho' não encontrado.

Neste caso, os erros foram enviados para o arquivo tempo.txt.



# Redirecionamento

- **Redirecionando stdout e stderr para um mesmo arquivo 2>&1**
  - fulano@pc01:~\$ find / > todosArquivos\_e\_erros.txt 2>&1
- Ordem dos redirecionamentos é importante
  - fulano@pc01:~\$ **ls > listadir 2>&1**
  - fulano@pc01:~\$ **ls > 2>&1 > listadir**
  - OBS.: No segundo comando o stderr não será direcionado para o arquivo listadir.

# Redirecionamento

- **Enviando dados para um comando: <**

```
fulano@pc01:~$ cat texto.txt
```

Terceira linha.

Quarta linha.

```
fulano@pc01:~$ tr 'r' 'R' < texto.txt
```

TeRceiRa linha.

QuaRta linha.

```
fulano@pc01:~$ cat texto.txt | tr '\n' ' '
```

Terceira linha. Quarta linha.

- OBS.: O comando `tr` traduz, comprime ou deleta caracteres da entrada padrão. Último comando substitui “nova linha” por “espaço”.

# Redirecionamento

- Enviando dados para um comando: <<
- Este redirecionamento serve principalmente para marcar o fim de exibição de um bloco. A entrada é redirecionada até encontrar um marcador que indique o final:

```
fulano@pc01:~$ cat texto.txt
```

Uma linha,  
outra linha.

```
fulano@pc01:~$ cat <<acabou >> texto.txt
```

Terceira linha.

Quarta linha.

acabou



Marcador  
indicando o final  
do bloco

```
fulano@pc01:~$ cat texto.txt
```

Uma linha,  
outra linha.

Terceira linha.

Quarta linha.

# Pipe |

- Envia a saída de um comando para a entrada do próximo comando para continuidade do processamento.
- Os dados enviados são processados pelo próximo comando que mostrará o resultado do processamento.

# Pipe

- Este comando faz a listagem longa de arquivos que é enviado ao comando `more` (que tem a função de efetuar uma pausa a cada 25 linhas do arquivo).
  - fulano@pc01:~\$ **ls -la | more**
  - fulano@pc01:~\$ **echo Victor | base64**  
VmljdG9yCg==
- **OBS.:** Comando base64 codifica um texto conforme descrito na RFC 3548

# Pipe

- **Diferença entre o "|" e o ">"**
- A principal diferença entre o "|" e o ">", é que o Pipe envolve processamento entre comandos, ou seja, a saída de um comando é enviado a entrada do próximo e o ">" redireciona a saída de um comando para um arquivo/dispositivo.